

Guía de Instalación Completa — Concert Ticketing System

Requisitos Previos

Software	Versión mínima	Comprobar
Python	3.10+	<code>python3 --version</code>
pip	22+	<code>pip --version</code>
Docker	20+	<code>docker --version</code>
Docker Compose	v2	<code>docker compose version</code>
NGINX (opc.)	1.18+	<code>nginx -v</code>

Nota: NGINX es opcional. Sin él, la arquitectura directa funciona igualmente en los puertos 8000/8001.

1. Descomprimir el proyecto

```
tar -xzf Practica_SD_COMPLETO.tar.gz
cd Practica_SD
```

2. Crear entorno virtual e instalar dependencias

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -e .
```

Esto instala todas las dependencias: `fastapi`, `uvicorn`, `redis`, `pika`, `httpx`, `pyyaml`, `pydantic`, `matplotlib`, `numpy`, `pandas`.

3. Configurar variables de entorno (opcional)

```
cp .env.example .env
# Editar .env si Redis/RabbitMQ no están en localhost
```

Variables principales:

Variable	Valor por defecto
REDIS_HOST	localhost
REDIS_PORT	6379
RABBITMQ_HOST	localhost
RABBITMQ_PORT	5672
RABBITMQ_USER	guest
RABBITMQ_PASSWORD	guest

4. Arrancar infraestructura (Redis + RabbitMQ)

```
docker compose -f tools/local_dev/docker-compose.yml up -d
```

Verificar que están arrancados:

```
docker ps    # Debe mostrar redis y rabbitmq healthy
```

5. Inicializar estado en Redis

```
PYTHONPATH=src python3 tools/local_dev/seed_state.py --ticket-type all
```

6. Ejecutar el sistema

Opción A: Todo junto (recomendado)

```
bash scripts/start_all.sh
```

Opción B: Solo arquitectura directa

```
bash scripts/start_direct_only.sh
```

Opción C: Solo arquitectura indirecta

```
bash scripts/start_indirect_only.sh 3 # 3 workers
```

7. Verificar que funciona

```
# API Directa (puerto 8000)
curl http://localhost:8000/health

# API Directa (puerto 8001)
curl http://localhost:8001/health

# Gateway Indirecto (puerto 8080)
curl http://localhost:8080/health

# NGINX (puerto 80, si disponible)
curl http://localhost/health
```

Respuesta esperada:

```
{"status": "healthy", "redis": "ok"}
```

8. Ejecutar tests unitarios

```
PYTHONPATH=src python3 -m pytest tests/unit/ -v
```

9. Ejecutar benchmarks

```
# Benchmark directo
bash scripts/run_direct_benchmark.sh

# Benchmark indirecto
bash scripts/run_indirect_benchmark.sh

# Análisis completo
bash scripts/run_full_analysis.sh
```

10. Parar todo

```
bash scripts/stop_all.sh
```

Estructura del Proyecto

```

Practica_SD/
├── config/           # Archivos YAML de configuración
├── deploy/           # NGINX, systemd, AWS
├── docs/             # Documentación de arquitectura
├── benchmarks/       # Ficheros de benchmark + outputs
├── scripts/          # Scripts de arranque, parada, benchmarks
├── src/concert_ticketing/
│   ├── shared/       # Config, logger, health, constantes
│   ├── core/         # Dominio, puertos, servicios
│   ├── adapters/     # Redis, RabbitMQ, REST API
│   ├── apps/         # Direct API, Gateway, Workers, Benchmark, Analysis
│   └── analysis/     # Re-exports para análisis
├── tests/            # Unit, integration, smoke, stress
├── tools/local_dev/  # Docker Compose + seed_state.py
└── pyproject.toml

```

Troubleshooting

Redis no conecta

```

docker compose -f tools/local_dev/docker-compose.yml up -d redis
docker logs $(docker ps -qf name=redis)

```

RabbitMQ no conecta

```

docker compose -f tools/local_dev/docker-compose.yml up -d rabbitmq
# Esperar ~15s para que RabbitMQ arranque completamente
docker logs $(docker ps -qf name=rabbitmq)

```

NGINX falla

NGINX es **opcional**. Sin él, accede directamente a `localhost:8000` o `localhost:8001`.
Si quieres usar NGINX:

```

sudo cp deploy/nginx/nginx.conf /etc/nginx/nginx.conf
sudo cp deploy/nginx/upstream.conf /etc/nginx/conf.d/upstream.conf
sudo nginx -t && sudo systemctl restart nginx

```

Puerto ya en uso

```

lsof -i :8000 # Ver qué proceso lo usa
bash scripts/stop_all.sh # Parar todo

```

Error “ModuleNotFoundError”

```

pip install -e . # Reinstalar el paquete

```